EWB CLOUD LABS | STUDENT PROJECT REPORT

AWS

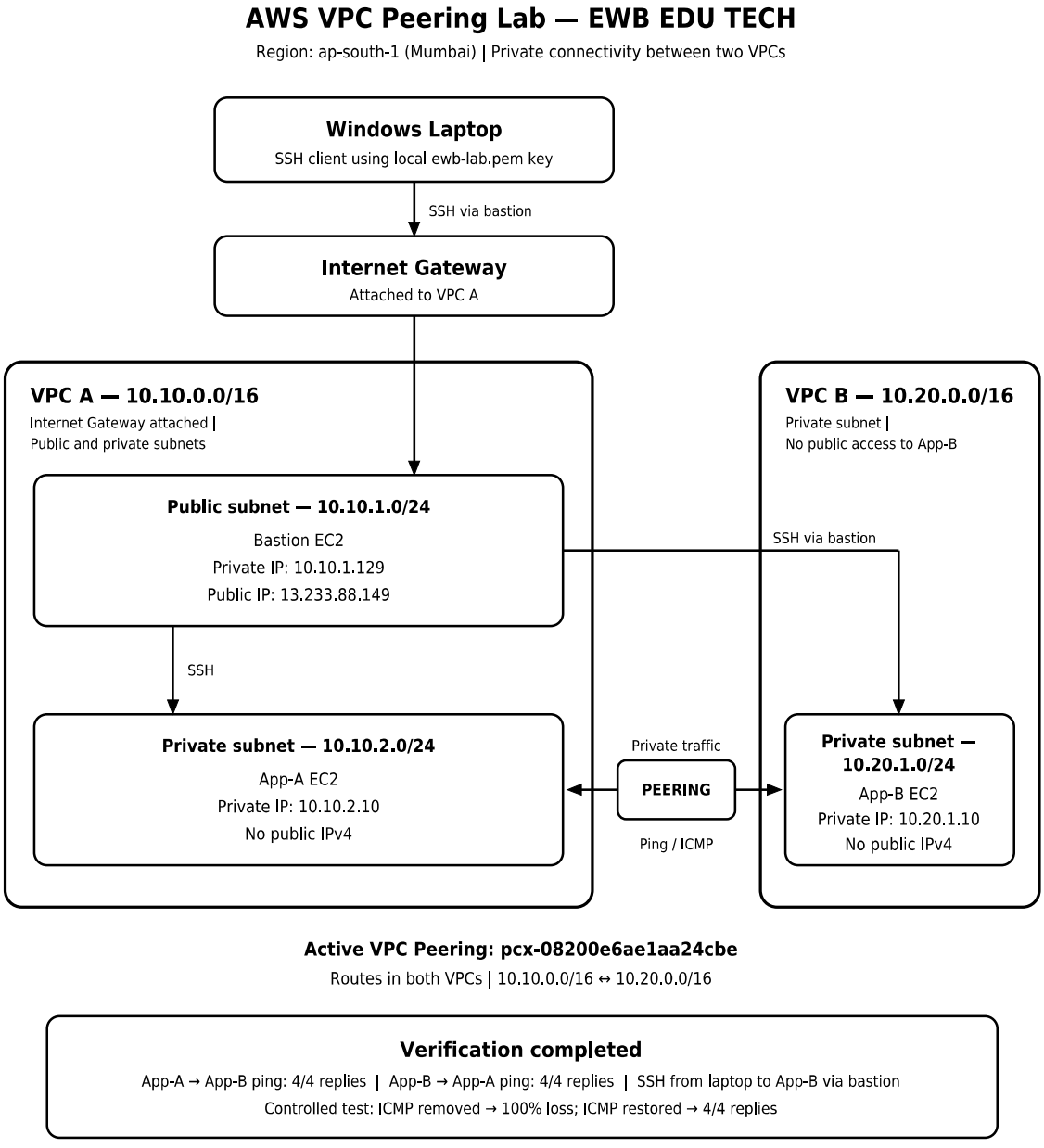
VPC Peering Project Report

Connect two VPCs. Prove private connectivity.

Name	Kandikatla Sailaja
Batch	AWS Internship – Batch 1
Programme	AWS Cloud Computing Internship – EWB Courses (EWB Edu Tech Pvt Ltd)
Project	Project 01 – AWS VPC Peering (2 VPCs, 3 EC2 instances, 1 peering connection)
Region	ap-south-1 (Mumbai)
Report date	26 October 2026

1. Architecture and design

This project connects two VPCs in ap-south-1 with a VPC peering connection so that App-A (in VPC A) and App-B (in VPC B) can talk to each other using private IPv4 addresses only. The bastion host in the public subnet of VPC A is the single entry point from the internet. Both private instances are administered by SSH ProxyJump through the bastion, and VPC B has no internet gateway.



Note: Peering provides private routing between VPCs; it does not provide VPC B with internet access through VPC A.

Administration path and traffic

Administration: laptop → bastion → App-A or App-B using SSH ProxyJump (the private key stays on the laptop). **Project traffic:** App-A ↔ App-B over private IPs through the peering connection. Peering does not give VPC B internet access through VPC A.

Why these CIDRs

10.10.0.0/16 (VPC A) and 10.20.0.0/16 (VPC B) do not overlap, which VPC peering requires. Each /24 subnet lies inside the range of its own VPC.

Address plan and resource inventory

Resource / Name tag	Location	CIDR / private IP	AWS ID
ewb-vpc-a	VPC A	10.10.0.0/16	vpc-0eb3a7706098e7110
ewb-a-public	VPC A subnet	10.10.1.0/24	subnet-0769cf8f819b2a8d2
ewb-a-private	VPC A subnet	10.10.2.0/24	subnet-052c6d9411135d9a7
ewb-vpc-b	VPC B	10.20.0.0/16	vpc-0a9f2293842e117a2
ewb-b-private	VPC B subnet	10.20.1.0/24	subnet-0d9a9a92b1c481eed
ewb-igw-a	Attached to VPC A	n/a	igw-06deee1691533f122
ewb-rt-a-public	VPC A	n/a	rtb-0bb846f4723b0f01c
ewb-rt-a-private	VPC A	n/a	rtb-05a1c48bb1d6d331b
ewb-rt-b-private	VPC B	n/a	rtb-0215f99ca98b45ec8
ewb-sg-bastion	VPC A	n/a	sg-06cacf3b93f151f2d
ewb-sg-app-a	VPC A	n/a	sg-06aa848aef879dfa9
ewb-sg-app-b	VPC B	n/a	sg-0889ff22e69c5d375
ewb-bastion	ewb-a-public	10.10.1.129; public IPv4 13.233.88.149	Instance ID not recorded in this report
ewb-app-a	ewb-a-private	10.10.2.10 (no public IP)	Instance ID not recorded in this report
ewb-app-b	ewb-b-private	10.20.1.10 (no public IP)	Instance ID not recorded in this report
ewb-a-b-peer	VPC A to VPC B	n/a	pcx-08200e6ae1aa24cbe
ewb-lab	EC2 key pair (RSA)	n/a	Key name only

The bastion public IPv4 address is temporary and is released when the instance is terminated during cleanup.

2. Subnets, internet gateway and route tables

Each subnet has an explicit route table association. The internet gateway is attached to VPC A only. The private subnets have no internet route and no public IPv4 address.

Route table ewb-rt-a-public (rtb-0bb846f4723b0f01c) – associated with ewb-a-public (subnet-0769cf8f819b2a8d2)

Destination	Target	Purpose
10.10.0.0/16	local	Bastion to App-A
0.0.0.0/0	igw-06deee1691533f122 (ewb-igw-a)	Bastion internet access
10.20.0.0/16	pcx-08200e6ae1aa24cbe (ewb-a-b-peer)	Bastion SSH to App-B through ProxyJump

Route table ewb-rt-a-private (rtb-05a1c48bb1d6d331b) – associated with ewb-a-private (subnet-052c6d9411135d9a7)

Destination	Target	Purpose
10.10.0.0/16	local	Existing VPC A route
10.20.0.0/16	pcx-08200e6ae1aa24cbe (ewb-a-b-peer)	App-A to VPC B

Route table ewb-rt-b-private (rtb-0215f99ca98b45ec8) – associated with ewb-b-private (subnet-0d9a9a92b1c481eed)

Destination	Target	Purpose
10.20.0.0/16	local	Existing VPC B route
10.10.0.0/16	pcx-08200e6ae1aa24cbe (ewb-a-b-peer)	Return path to VPC A

Why both sides need a route: a packet from App-A reaches VPC B only if the App-A subnet's route table points 10.20.0.0/16 at the peering connection, and App-B's reply only comes back if the VPC B route table points 10.10.0.0/16 at it. The bastion's subnet also needs the 10.20.0.0/16 route because ProxyJump makes the bastion open the SSH connection to App-B itself. The more specific 10.20.0.0/16 route wins over the 0.0.0.0/0 internet route.

3. Peering connection and security groups

Field	Value
Name / ID	ewb-a-b-peer / pcx-08200e6ae1aa24cbe
Requester VPC	ewb-vpc-a (vpc-0eb3a7706098e7110) – 10.10.0.0/16
Accepter VPC	ewb-vpc-b (vpc-0a9f2293842e117a2) – 10.20.0.0/16
Account / Region	Same account / ap-south-1 (Mumbai)
Status	Active (request accepted)

Inbound security group rules

Security group	Type / port	Source
ewb-sg-bastion sg-06cacf3b93f151f2d (VPC A)	SSH / TCP 22	Student's public IPv4 /32
ewb-sg-app-a sg-06aa848aef879dfa9 (VPC A)	SSH / TCP 22	10.10.1.129/32 (bastion private IP)

Security group	Type / port	Source
ewb-sg-app-a sg-06aa848aef879dfa9 (VPC A)	Custom ICMP – IPv4, Echo request	10.20.1.10/32 (App-B)
ewb-sg-app-b sg-0889ff22e69c5d375 (VPC B)	SSH / TCP 22	10.10.1.129/32 (bastion private IP)
ewb-sg-app-b sg-0889ff22e69c5d375 (VPC B)	Custom ICMP – IPv4, Echo request	10.10.2.10/32 (App-A)

Outbound rules were left at the default allow-all and the default network ACLs were used. Security groups are stateful, so replies to permitted traffic are allowed automatically. SSH and ICMP are never opened to 0.0.0.0/0. The bastion's private IP in this deployment is 10.10.1.129.

4. Test results

Results below are the outcomes obtained during the lab.

#	Test	Where / command	Result
01	Baseline (before peering)	App-A: `ping -c 4 10.20.1.10`	**CAPTURED.**
02	Forward path	App-A, after routes: `ping -c 4 10.20.1.10`	4/4 replies (success)
03	Reverse path	App-B, after routes: `ping -c 4 10.10.2.10`	4/4 replies (success)
04	Private SSH to App-B	Laptop: `ssh ewb-app-b` (ProxyJump via bastion)	Shell on App-B obtained through the bastion
05	Controlled failure	Echo request rule (from 10.10.2.10/32) removed from ewb-sg-app-b; ping from App-A	100% packet loss
06	Recovery	Same rule restored; ping from App-A repeated	4/4 replies (success)

Forward path and return path

Forward path: App-A (10.10.2.10) sends an echo request to 10.20.1.10. The route table of ewb-a-private matches 10.20.0.0/16 and sends the packet to the peering connection, which delivers it into VPC B. The security group of App-B allows ICMP echo requests from 10.10.2.10/32, so App-B accepts it.

Return path: App-B replies to 10.10.2.10. The route table of ewb-b-private matches 10.10.0.0/16 and sends the reply back through the same peering connection. Because security groups are stateful, the reply is allowed without a separate inbound rule on App-A for that reply. The reverse ping (App-B to App-A) works the same way, using the ICMP rule on ewb-sg-app-a that allows 10.20.1.10/32.

Controlled ICMP failure and recovery

Only the ICMP echo rule on ewb-sg-app-b was removed; the SSH rule, the peering connection and all routes were left unchanged. The ping from App-A then showed 100% packet loss. After the exact rule was restored, the ping returned 4/4 replies. This shows that an Active peering connection and correct routes are necessary but not sufficient: the destination security group must also permit the traffic.

5. Cleanup

Status: Completed terminating/detaching.

Step	Action	Status
1	Terminate ewb-bastion, ewb-app-a, ewb-app-b; confirm lab EBS volumes are deleted; release any Elastic IP; remove the lab key pair	Done
2	Delete peering routes and ewb-a-b-peer; delete security groups	Done
3	Delete subnets and custom route tables; detach and delete ewb-igw-a; delete both VPCs	Done
4	Verify EC2, volumes, public IPv4 and VPC resources in ap-south-1 are clear; check Billing	Done

6. What I learned

- VPC peering is only the link between two VPCs. Traffic flows only when both route tables and the security groups allow it.
- The return route matters as much as the forward route, because replies need a path back to the sender.
- A bastion host in a public subnet with ProxyJump lets me manage private instances without a public IP on them and without copying the private key onto any server.
- Peering is not transitive and does not share an internet gateway, so VPC B stays private even though VPC A has internet access.